



Práctica 4. El API de Java

Objetivos de aprendizaje

1. Consultar y comprender la documentación del API de Java.	Ser capaz de buscar, leer y entender la documentación oficial de las clases estándar (Random, Character, String, ArrayList, Collections) y aplicar ese conocimiento en el desarrollo.
2. Entender porque el programa se puede extender gracias al diseño previo de clases y relaciones.	Saber aprovechar la estructura de clases y relaciones creada previamente para añadir nuevas funciones o juegos de forma sencilla, aplicando el polimorfismo para gestionar distintos tipos de juegos o comportamientos bajo una misma interfaz o clase base.
3. Saber gestionar colecciones y datos externos:	Saber leer y manipular colecciones dinámicas y datos externos (como ficheros) para hacer el sistema más flexible y ampliable.
4. Convenciones de codificación en Java	Aplicar adecuadamente las normas de estilo y convenciones de codificación de Java (nombres de variables, formato, comentarios, indentación, etc.), mostrando código claro y legible.

Descripción

En esta práctica se trabajará, a partir del código de la práctica anterior, con el API de Java. En este sentido será importante acceder, leer y entender la documentación que recoge la página web de Oracle donde se consigna toda esta información. En particular, será necesario acceder a las clases Random, Character, String, Collections y ArrayList del API de Java.

1. Primera parte. Clases Random y ArrayList

Consultando la documentación del API de java, usar las clases Random y ArrayList del paquete java.util para que el número que haya que adivinar en los juegos de números sea un número aleatorio en lugar de un número predefinido por el programador.

1.1. Modificación de la Clase JuegoAdivinaNumero

- Redefinir el método **reiniciaPartida** para que, además de ejecutar el código definido en la clase Juego, asigne un valor aleatorio al dato miembro que contiene el número a adivinar. Una opción para abordar la redefinición del método **reiniciaPartida** de forma que genere el número a adivinar de manera polimórfica (dependiendo del objeto que lo invoque) es crear un nuevo método denominado **numAleatorio** que es invocado desde **reiniciaPartida** y que retorna un número entero de forma aleatoria. Este método deberá ser implementado en

la clase **JuegoAdivinaNumero** y sobrescrito en sus dos subclases, atendiendo a si el número generado aleatoriamente es par o impar.

- Para que el método **numAleatorio** genere un número entero aleatorio se usará un objeto de tipo **Random** (java.util.Random). A la hora de construir este objeto, es conveniente pasarle una semilla que evite que se genere siempre la misma secuencia de números pseudoaleatorios. Para ello puede usarse el método `System.currentTimeMillis()`.
- Dadas las anteriores modificaciones el constructor ya no tendrá como parámetro el número por adivinar.

1.2. **Modificación de Clase JuegoAdivinaNumeroPar**

- Redefine el método **numAleatorio** en esta clase para que genere un número par aleatorio entre 0 y 10.

1.3. **Modificación de la Clase JuegoAdivinaNumeroImpar**

- Redefine el método **numAleatorio** en esta clase para que genere un número impar aleatorio entre 0 y 10.

1.4. **Modificación de la clase JuegosReunidos:**

- Se sustituye el array de objetos de tipo Jugable por un ArrayList de objetos de tipo **Jugable** (ArrayList<Jugable>).
- Se cambia el constructor para que inicialice el ArrayList.
- Se reemplaza el método **numeroJuegos** por el método **listadoJuegos** que no recibe parámetros y que devuelva la lista de juegos de manera inmodificable.
- Se deberán adaptar los métodos **agregar** y **recuperarJuego** para que realicen correctamente su tarea con esta nueva estructura. Para ello se deberá tener en cuenta que, ahora, el método **agregar** solo tendrá un único parámetro de entrada, el objeto de tipo Jugable.

1.5. **Modificación de la clase Menu:**

- Al haber reemplazado el método **númeroJuegos** por **listadoJuegos** se deberán modificar los métodos **mostrarOpciones** y **chequeoOpcion** para que usen este nuevo método en lugar del anterior.

2. **Segunda parte. El juego del Ahorcado Inglés.**

Haciendo uso de las clases Character, String, ArrayList y Random, se va a implementar un nuevo juego, el Juego del Ahorcado Inglés, esta vez no basado en números sino en palabras tomadas del inglés y formadas por letras minúsculas.

2.1. **Creación de la Clase JuegoAhorcadoIngles**

- Se Deriva de la clase JuegoConVidas.
- Implementará la interfaz Jugable.
- Tendrá tres ArrayList como atributos:
 - Dos de caracteres, uno para almacenar la palabra a adivinar y otro, denominado guiones, donde se almacenarán los caracteres acertados.
 - El tercero es un ArrayList de String que contendrá todas las palabras de un diccionario y se utilizará para seleccionar de forma aleatoria la palabra a adivinar.
- El constructor tomará como primer parámetro el número de vidas y como segundo un ArrayList de String. Este ArrayList se utilizará para inicializar el atributo que contiene el diccionario de palabras.

- Tendrá los siguientes métodos:
 - Los métodos públicos **muestraNombre** y **muestraInfo** reescritos para mostrar respectivamente el nombre y la información de este nuevo juego.
 - El método booleano **existeCaracter** recibe como parámetro un carácter y comprueba si este se encuentra presente, una o más veces, en la palabra a adivinar. Para ello, recorre el ArrayList que contiene la palabra y, cada vez que encuentra una coincidencia, incrementa un contador y sustituye el guion correspondiente en el ArrayList de guiones por el carácter acertado. Una vez finalizado el recorrido, devuelve true si el contador es mayor que cero, indicando que el carácter aparece en la palabra, o false en caso contrario. Este método es privado
 - El método privado **mostrarArray** muestra por pantalla el contenido del ArrayList de caracteres que se le pasa por parámetro.
 - Reescritura del método **reiniciaPartida**: Para ello se invoca primero el método **reiniciaPartida** del supertipo. A continuación, vacía los ArrayList que almacenan la palabra a adivinar y los guiones, y selecciona de forma aleatoria una palabra del diccionario de palabras. Esta palabra se utiliza para inicializar el ArrayList de la palabra a adivinar y rellenar el ArrayList de guiones con el carácter '-'. Por último, se muestra por pantalla la palabra seleccionada (solo con fines de comprobación) y el contenido del ArrayList de guiones, que revela al usuario el número de caracteres que tiene la palabra.
 - Implementa el método booleano **juega**:
 - ✓ Captura en una variable local el primer carácter de la cadena de caracteres (String) recibido como parámetro.
 - ✓ Se invoca el método **existeCaracter**, pasándole como parámetro el carácter obtenido previamente. El valor de retorno de este método se utiliza para controlar una estructura selectiva: si **existeCaracter** devuelve true, se mostrará por pantalla el ArrayList de guiones, revelando las posiciones acertadas, y se comprobará si la palabra ha sido adivinada. Para dicha comprobación, se emplea otra estructura selectiva anidada: si la palabra está completa, se actualiza el récord y la partida finaliza retornando false; si aún no se ha adivinado, el método devuelve true para continuar el juego. Si **existeCaracter** devuelve false, se llamará al método **quitaVida**, cuyo valor de retorno permitirá determinar si al jugador le quedan vidas para seguir jugando. En caso afirmativo, se mostrará el mensaje *"Inténtelo de nuevo"* junto con el ArrayList de guiones. Este mismo valor será el que se devuelva finalmente como resultado del método **juega**.

2.2. Modificación de la clase MyInput

- Se incluirá un nuevo método denominado **leeFichero** (se incluye el código) que recibe como parámetro una cadena de caracteres (String) que representa el camino, o "path", donde se encuentra el fichero de texto plano y retorna como resultado un ArrayList que contiene en cada nodo todas las palabras contenidas en dicho fichero. Por esta razón, el fichero de texto debe estar configurado de tal forma que cada una de sus líneas solo contiene una palabra del diccionario. El método devuelve su contenido en un ArrayList que contiene en cada nodo una palabra.

- El fichero de texto se debe almacenar en la carpeta raíz del proyecto. De esta forma no es necesario pasar toda la ruta para su correcta localización.

```
public static ArrayList <String> leeFichero(String pathFichero){
    ArrayList <String> v = new ArrayList <String>();
    File fichero=null;
    FileReader fr=null;
    BufferedReader br=null;
    try{
        fichero=new File(nombreFichero);
        fr=new FileReader(fichero);
        br=new BufferedReader(fr);
        String linea;
        while ((linea=br.readLine())!=null){
            v.add(linea);}
    }
    catch (Exception e){
        e.printStackTrace();
    }
    finally {
        try {
            if (null!= fr){
                fr.close();
                br.close();}
        }
        catch (Exception e1){
            e1.printStackTrace();
        }
    }
    return v;
}
```

2.3. Modificación de la clase principal

Por último, se deberá modificar la clase Aplicación para que desde el método principal se cree:

- Se crea un objeto de tipo **JuegosReunidos**, al que se le añadirán los juegos a los que se quiera jugar (Los juegos así creados y almacenados tendrán 5 vidas), incluido el juego del ahorcado, al cual habrá que pasarle por parámetro el ArrayList que contiene el diccionario de palabras, que se encuentra almacenado en un fichero externo.
- Un objeto de tipo **Menu** al que se le pasará por parámetro la referencia que apunta al objeto de tipo **JuegosReunidos**.
- Por último, la invocación a través del objeto de tipo **Menu** del método **ejecuta**.

3. Resultados Evaluación y retroalimentación

3.1. Rúbrica de logro de resultados de aprendizaje

Criterio	Consigue el resultado	No consigue el resultado
1. Consulta y comprensión de la documentación del API de Java	Busca, interpreta y aplica correctamente la documentación oficial de las clases estándar del API de Java (Random, Character, String, Collections, ArrayList) para resolver problemas y añadir funcionalidades nuevas, justificando su uso con ejemplos concretos del código.	No consulta, interpreta o aplica correctamente la documentación del API de Java, o no justifica su uso en el desarrollo del código.
2. Extensibilidad basada en el diseño de clases y relaciones	Aprovecha la estructura de clases y relaciones existente para añadir nuevas funcionalidades o juegos, explicando cómo el diseño previo y el uso de polimorfismo facilitan la ampliación del sistema.	No aprovecha el diseño de clases o el polimorfismo para extender el programa, o no comprende cómo el diseño previo facilita la ampliación.
3. Gestión de colecciones y datos externos	Utiliza correctamente colecciones dinámicas (ArrayList) y sabe gestionar datos externos (por ejemplo, ficheros de palabras), explicando su función y aportando ejemplos de mejora o ampliación del sistema respecto a soluciones anteriores.	No utiliza adecuadamente las colecciones o la gestión de datos externos, o no comprende su función en la mejora y ampliación del sistema.
4. Convenciones de codificación en Java	Aplica correctamente las normas de estilo y convenciones de codificación de Java (nombres, formato, comentarios, indentación), mostrando código claro y legible y justificando sus decisiones.	No aplica correctamente las normas de estilo Java o no demuestra comprender su importancia para la legibilidad y el mantenimiento del código.

3.2. Preguntas de autoevaluación

Responde a las siguientes preguntas, justificando tus respuestas con fragmentos de código o ejemplos concretos desarrollados en la práctica:

1. ¿Cómo te ha ayudado consultar la documentación del API de Java para resolver dudas o implementar funcionalidades nuevas en esta práctica?. Explica algún caso concreto.
2. ¿Por qué el programa se puede extender fácilmente gracias al diseño de clases y relaciones?. Pon un ejemplo de cómo has añadido una nueva función o juego aprovechando la estructura existente.
3. ¿Cómo has utilizado el polimorfismo para gestionar distintos tipos de juegos o comportamientos?. Da un ejemplo de cómo una misma interfaz o clase base te ha permitido trabajar con juegos diferentes de manera uniforme.
4. ¿De qué manera has utilizado clases del API como Random, Collections o ArrayList, y cómo ha influido su uso en tu aplicación? Explica su función y las ventajas que has observado respecto a soluciones anteriores.
5. ¿Cómo gestionas la incorporación de datos externos (por ejemplo, palabras para el ahorcado) y ¿qué aporta al sistema el uso de colecciones dinámicas?
6. ¿Qué convenciones de codificación en Java has seguido al realizar ampliaciones o cambios en el código?. Da ejemplos sobre el uso de nombres de variables, formato, comentarios y organización de clases para facilitar la comprensión y el mantenimiento del programa.

3.3. Obteniendo retroalimentación de la IAG

A continuación, deberás solicitar a la IAG con la que trabajes una primera evaluación (retroalimentación) sobre la corrección de tu código y las respuestas que has dado a las preguntas planteadas. Para ello elabora un prompt lo más detallado posible que permita a la IAG realizar una evaluación de la corrección tanto del código como de las respuestas proporcionadas. Es importante que en tu prompt incluyas:

- El enunciado de la práctica
- Los objetivos de aprendizaje
- La rúbrica de logro de resultados de aprendizaje.
- El código desarrollado
- Las preguntas de autoevaluación junto con tus respuestas

La valoración recibida **no debe ser una nota numérica**, sino un análisis en el que se indiquen:

- Qué aspectos están bien resueltos.
- Qué errores o carencias se detectan.
- Qué aspectos son mejorables.
- Si se han alcanzado los objetivos y resultados de aprendizaje previstos, tanto en el código como en las respuestas teóricas.

Es fundamental que tu prompt recoja con claridad todos los aspectos mencionados, de modo que la IAG oriente su análisis exactamente a lo que se espera en la evaluación. Si la respuesta de la IAG no es suficientemente clara, ajustada o útil, mejora tu prompt añadiendo explicaciones adicionales, ejemplos concretos o aclaraciones sobre lo que necesitas.

Recuerda: **la calidad de la retroalimentación recibida depende de la precisión y el contexto que incluyas en tu consulta.** Compara siempre la respuesta de la IAG con los materiales de clase u otras fuentes fiables, o consulta con el/la profesor/a en caso de dudas persistentes. Esto te ayudará a desarrollar un uso crítico y responsable de la IAG.

4. Informe que debe subirse al campus virtual de la asignatura

Una vez obtengas una respuesta satisfactoria, sube a la tarea correspondiente en Moodle un informe en PDF, **utilizando para ello la plantilla que está disponible en el campus virtual.** Este informe incluye:

- Nombre y apellidos
- La IAG utilizada, su versión (si procede) y la fecha.
- El desarrollo de la práctica: El código de la práctica y las respuestas a las preguntas que se formulan en esta práctica.
- Interacción con la IAG: Transcribe el diálogo íntegro y ordenado (tus prompts y las respuestas de la IAG).
- Una valoración personal sobre la utilidad y limitaciones de la retroalimentación recibida de la IAG, indicando qué mejoras podrás incorporar gracias a ella (si las hubiera) y reflexionando brevemente sobre su impacto en tu aprendizaje.
- La declaración de autoría y uso de IAG, conforme a lo indicado en el siguiente apartado.

Puedes aprovechar las observaciones de la IAG para mejorar el código o tus respuestas en la práctica siguiente.

Nota: Es obligatorio entregar el informe en PDF utilizando la plantilla que se facilita en el campus virtual de la asignatura y completar correctamente todos sus apartados; cualquier entrega que no se ajuste a esta plantilla —por ejemplo, subir solo el código o un PDF que no se corresponda con la plantilla— se considerará no válida y conllevará el suspenso de toda la práctica guiada (las cinco entregas).

5. Uso de la IAG, declaración de autoría y buenas prácticas

El uso de herramientas de IAG está permitido en esta práctica únicamente para obtener retroalimentación y sugerencias de mejora sobre el trabajo realizado (por ejemplo, evaluación de código, propuestas de corrección o ayuda en la redacción final). No se permite la elaboración inicial de código, respuestas o informes previos mediante IAG. Todo el desarrollo y resolución de la práctica debe ser realizado personalmente por el estudiante.

Para asegurar la transparencia, la responsabilidad y el uso adecuado de la IAG, la plantilla para la elaboración del informe incluye un apartado final de “Declaración de autoría y uso de IAG”. Mediante esta declaración, el estudiante manifiesta que el trabajo presentado ha sido realizado por él/ella, que asume la responsabilidad sobre su contenido y sobre las decisiones adoptadas durante su elaboración, y que la información proporcionada en el informe acerca del uso de herramientas de Inteligencia Artificial Generativa es completa y veraz.

La entrega del informe supone la aceptación expresa de esta declaración y la asunción de responsabilidad sobre el trabajo presentado.

Recuerda: El uso responsable de la IAG contribuye al aprendizaje autónomo y a la mejora continua, pero no puede sustituir al desarrollo personal de competencias técnicas y críticas. El falseamiento de la autoría será considerado una mala praxis académica.